

# Business Entity Resolution: Approach & Methodology

Amazon ML Challenge 2026 · Team Gaya

**Submitted by** Chaitanya Ram S (Team Gaya)

**Date** 2 October 2026

**Code** [github.com/theLeverager1705/Team-Gaya-ER-Challenge-2026](https://github.com/theLeverager1705/Team-Gaya-ER-Challenge-2026) (submitted pipeline: `code/business_entity_resolution/`)

**0.9264**

Macro F0.5 on 30,057 held-out training entities (never used for fitting or selection)

**0.902**

Public leaderboard macro F0.5 (v2; test includes France, unseen in training)

**3.84**

Candidate pairs per Source 1 entity on test (6.65M pairs out of  $\sim 1.7 \times 10^{13}$  possible)

**0.980 / 0.854**

Pair precision / pair recall on the held-out split

**In one paragraph.** Candidates come from IDF-weighted retrieval over rare *name* and *address* keys inside each country, computed as chunked sparse-matrix products so the whole corpus fits on one laptop. A two-stage LightGBM model scores each candidate pair: stage 1 uses 26 pairwise features; stage 2 adds 12 *group-context* features that compare each candidate with the entity's other candidates, because the true matches of one business resemble each other. A learned pre-filter shrinks the candidate set from 30 to about 3.8 per entity at no measurable cost, and a decision rule tuned directly for macro F0.5 turns probabilities into match lists.

## 1. Methodology

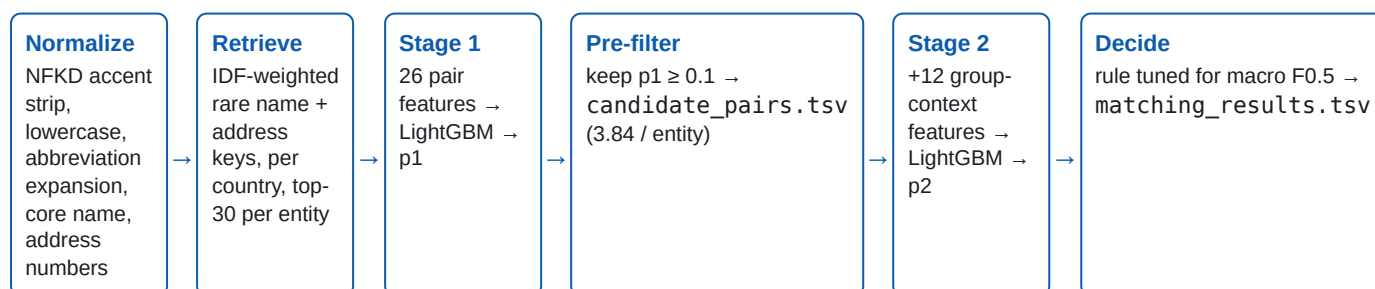
### 1.1 Problem and metric

For every Source 1 business, return all Source 2 / Source 3 records that describe the same real-world business. Scoring is macro F0.5 per Source 1 entity, with singletons included (a singleton scores 1 only for an empty prediction). F0.5 weights precision above recall, so the pipeline is tuned to avoid wrong merges.

### 1.2 What the data told us

- **Scale.** 2.21M training and 1.73M test Source 1 entities, each split with about 10M Source 2+3 records. All-pairs comparison is impossible, so blocking has to use inverted indexes.
- **Labels.** 5.6% of training entities are singletons; the rest have 1–11 matches (mean 3.7). In a 5,000-entity sample, all 17,362 true pairs share the country label, so country is a safe partition key.
- **Noise.** Source 2 names are often written in native script (Bengali, Devanagari); unrelated trade/DBA names appear at the same address (*Syndrex* ↔ *Straight Edge Pub*, both at 515 Monroe Ave); injected diacritics (*Cáble*, *Prívate*); character swaps (*Co1onial*); word-order changes; legal-suffix churn (*Pvt Ltd* / *Private Limited*); abbreviated or reordered addresses; perturbed street numbers (26 vs 26/6).
- **Consequence:** the address is as important as the name, and name-only matching fails.
- Up to 349 lines per file contain a literal `"`, so files are read with quote handling disabled (`QUOTE_NONE`; every line has exactly 4 tab-separated fields).

### 1.3 Pipeline



### 1.4 Training and validation protocol

- **Hold-out by hash.** About 10% of training Source 1 entities ( $\text{CRC32 of the id mod } 10 = 0$ ) are excluded from training altogether and used only for end-to-end checks of the full pipeline.

- **Training sample.** 300,000 Source 1 entities sampled from the rest (seed 42), blocked against the *full* training Source 2/3 corpus so candidate lists look exactly like test-time lists. This produced 8.99M labelled candidate pairs.
- **Entity-level split** of that sample: 80% *fit* / 10% *tune* (early stopping and every rule or cutoff choice) / 10% *report* (never used for any decision; all reported scores come from here).
- **Out-of-fold stage 1.** Within *fit*, two folds produce out-of-fold stage-1 probabilities, so stage 2 learns from probabilities of the same quality it sees at inference.
- Fully deterministic: fixed seeds, LightGBM `deterministic=True`, murmur3 key hashing.

## 2. Candidate Generation / Blocking Strategy

### 2.1 Normalization

- NFKD accent stripping (*Cáble* → *cable*), lowercase, apostrophes removed, all other punctuation to spaces.
- Generic abbreviation expansion applied identically to every record: street types (*rd* → *road*, *blvd*, *hwy* ...), units (*ste*, *apt*, *fl*), directions (*n*, *ne* ...), Indian address words (*ngr* → *nagar*, *opp*, *nr*) and legal forms (*pvt* → *private*, *ltd* → *limited*, *corp*, *inc*).
- **Core name:** the normalized name minus legal forms and function words (*private*, *limited*, *llc*, *gmbh*, *sarl*, *sas*, *the*, *and*, *de*, *la* ...), so *Pvt Ltd* / *Private Limited* churn disappears.
- Derived fields: sorted unique address numbers, first address number (usually the street number), core-name token count, and the share of non-ASCII characters in the raw name (native-script detector).

### 2.2 Step 1: IDF-weighted rare-key retrieval

- **Keys.** Core-name tokens (length > 1), core-name bigrams, address tokens (length > 1, or any number), and address bigrams, prefixed by type and hashed into  $2^{24}$  buckets (scikit-learn `HashingVectorizer`, binary, no vocabulary in memory).
- **Rare keys only.** Keys occurring in more than 1,000 Source 2/3 records of the partition are dropped as non-discriminative.
- **Score.** For Source 1 record  $q$  and Source 2/3 record  $c$  in the same country, with  $N$  the partition size and  $df(k)$  the key's document frequency:

$$\text{score}(q, c) = \sum_{k \in K_{\text{name}}(q) \cap K_{\text{name}}(c)} \text{idf}(k) + \sum_{k \in K_{\text{addr}}(q) \cap K_{\text{addr}}(c)} \text{idf}(k), \quad \text{idf}(k) = \log(1 + N / df(k)) \text{ if } df(k) \leq 1000, \text{ else } 0$$

- **Computation.** For each country, Source 2/3 is indexed as a sparse key × record matrix; Source 1 rows are IDF-weighted query vectors. Scores for 5,000 Source 1 rows at a time are one sparse matrix product (name and address separately, then summed), followed by a vectorised top-K per row. The name and address parts of the score are kept and reused as features.
- **Country as an open set.** Partitions are whatever labels appear in Source 1, so France (absent from training) needs no special handling. Country is never one-hot encoded or used as a model feature.
- **Why addresses:** address keys recover true matches whose name is in a native script or is an unrelated trade name, where the name shares nothing with Source 1.

### 2.3 Choosing K (recall at full corpus scale, 20k entities)

| Top-K retrieved              | True pairs kept |
|------------------------------|-----------------|
| 5                            | 79.2%           |
| 20                           | 90.4%           |
| <b>30 (chosen, the knee)</b> | <b>91.5%</b>    |
| 60                           | 93.0%           |

On the 300k training sample, top-30 retrieval gave 29.96 pairs per entity and a recall ceiling of 91.4%.

### 2.4 Step 2: learned pre-filter

The stage-1 model scores all 30 retrieved pairs, and only pairs with  $p1 \geq 0.1$  stay candidates. The cutoff is the largest value in  $\{0, 0.001, 0.003, 0.01, 0.02, 0.05, 0.1\}$  whose *tune*-split macro F0.5 is within 0.0001 of using no cutoff. The final (stage-2) model scores only these pairs, and they are exactly `candidate_pairs.tsv`; matches are always a subset.

| p1 cutoff (report split)              | Candidates / entity | Recall ceiling | Macro F0.5    |
|---------------------------------------|---------------------|----------------|---------------|
| none                                  | 29.95               | 91.37%         | 0.9264        |
| 0.001                                 | 5.49                | 91.35%         | 0.9264        |
| 0.01                                  | 4.31                | 91.22%         | 0.9264        |
| 0.05                                  | 3.80                | 90.85%         | 0.9263        |
| <b>0.1 (chosen on the tune split)</b> | <b>3.63</b>         | <b>90.42%</b>  | <b>0.9263</b> |

**Final candidate set on test:** 51.9M retrieved pairs → **6,652,763 candidate pairs (3.84 per Source 1 entity)**, against  $\sim 1.7 \times 10^{13}$  possible pairs (a reduction of more than 99.99999%). 56,812 entities have no candidate and are predicted as singletons. For transparency: the stage-1 model scores all 30 retrieved pairs, and the stage-2 group features summarize those stage-1 scores; only the final model's input is restricted to the pre-filtered candidates.

## 3. Model Architecture and Feature Engineering

### 3.1 Stage-1 pair features (26)

All string similarities are computed with `rapidfuzz.process.cpdist` over whole arrays of pairs (C++, multithreaded), not per-pair Python calls, because the test set has tens of millions of pairs.

| Group            | Features                                                                                                                                                    |
|------------------|-------------------------------------------------------------------------------------------------------------------------------------------------------------|
| Blocking (7)     | Total IDF score, name part, address part; rank among the entity's candidates; number of candidates; gap to and ratio of the entity's best score             |
| Name (7)         | On core names: ratio, token-set ratio, token-sort ratio, partial ratio, Jaro-Winkler; gap to the entity's best name token-set score; either-name-empty flag |
| Address (4)      | Ratio, token-set ratio, token-sort ratio on normalized addresses; gap to the entity's best address token-set score                                          |
| Numbers (3)      | Token-set ratio on the sorted address-number strings; first-number equality; first-number missing flag                                                      |
| Record shape (5) | Core-name token counts (both sides); non-ASCII share of each raw name (native-script detection); Source 3 flag                                              |

### 3.2 Stage-2 group-context features (+12)

True matches of one business resemble each other, so each candidate is judged against its entity's other candidates. The *reference* is the entity's best *other* candidate by p1.

| Group            | Features                                                                                                                         |
|------------------|----------------------------------------------------------------------------------------------------------------------------------|
| Own position (3) | Stage-1 probability p1; its rank within the entity; gap to the entity's best p1                                                  |
| Competition (4)  | Max and sum of the other candidates' p1; sum of other same-source candidates' p1; number of candidates with $p1 \geq 0.5$        |
| Reference (5)    | Reference's p1; same-source flag; name, address and address-number token-set similarity between this candidate and the reference |

Example: a Source 2 record with an unrelated trade name gets little support from stage 1, but if it shares the address and numbers of a high-confidence candidate, stage 2 recovers it.

### 3.3 Models

|                              | Stage 1                                                                                                                                                                                     | Stage 2                                                                            |
|------------------------------|---------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|------------------------------------------------------------------------------------|
| Algorithm                    | LightGBM binary classifier (MIT licence)                                                                                                                                                    |                                                                                    |
| Hyperparameters              | 127 leaves, learning rate 0.1, min_child_samples 100, feature_fraction 0.8, bagging_fraction 0.8 (every iteration), L2 = 1.0, up to 1,500 rounds with early stopping (50) on the tune split |                                                                                    |
| Inputs                       | 26 pair features                                                                                                                                                                            | 26 + 12 = 38 features (trained on out-of-fold p1)                                  |
| Trees (after early stopping) | 1,067                                                                                                                                                                                       | 571                                                                                |
| Model file size              | 15 MB                                                                                                                                                                                       | 8 MB                                                                               |
| Top features (share of gain) | rank among candidates 47%, address-number token-set 15%, address token-set 6%, relative blocking score 5%, name Jaro-Winkler 4%                                                             | p1 67%, p1 rank 19%, candidates with $p1 \geq 0.5$ 4%, address-number token-set 3% |

Total model size is a few MB, far below the 8B-parameter limit. No pretrained language models or embeddings are used.

### 3.4 Decision rule (tuned for macro F0.5)

Two rule families are grid-searched on the tune split for each stage, and the (stage, rule) with the best tune score is kept:

- **Threshold rule:** keep every candidate with  $p \geq t_{\text{all}}$ , plus the entity's single best candidate if  $p \geq t_{\text{top}}$ . Only 5.6% of entities are singletons and an empty prediction scores 0 for everyone else, so a lower bar for the top candidate pays off.
- **Expected-F0.5 rule:** per entity, with candidates sorted by  $p$ , predict the top  $j$  that maximizes  $EF(j) = 1.25 \cdot \sum_{i \leq j} p_i / (j + 0.25 \cdot (\sum_i p_i + c))$ , where  $c$  estimates true matches that blocking never retrieved; predict nothing when  $s \cdot \prod_i (1 - p_i)$  (an estimate that the entity is a singleton) is larger.

**Chosen:** stage 2 with the threshold rule,  $t_{\text{all}} = 0.725$ ,  $t_{\text{top}} = 0.56$ , applied to the pre-filtered candidates.

| Model              | Rule (tuned parameters)                 | Tune split    | Report split  |
|--------------------|-----------------------------------------|---------------|---------------|
| Stage 1 only       | threshold (0.675 / 0.64)                | 0.9164        | 0.9160        |
| Stage 1 only       | expected F0.5 ( $c = 0.8$ , $s = 1.0$ ) | 0.9166        | 0.9171        |
| <b>Stage 1 + 2</b> | <b>threshold (0.725 / 0.56)</b>         | <b>0.9269</b> | <b>0.9264</b> |
| Stage 1 + 2        | expected F0.5 ( $c = 0.0$ , $s = 0.9$ ) | 0.9267        | 0.9265        |

The group-context stage adds about one point of macro F0.5. Once probabilities are this good, the two decision rules tie.

## 4. Other Relevant Information

### 4.1 Results and submission history

| Version   | Change                                                                   | Candidates / entity | Report F0.5   | Public LB |
|-----------|--------------------------------------------------------------------------|---------------------|---------------|-----------|
| v1        | Stage-1 LightGBM (200k training entities), threshold rule                | 30                  | 0.9165        | –         |
| v2        | + stage-2 group-context model (300k entities), rule chosen on tune split | 30                  | 0.9264        | 0.902     |
| <b>v3</b> | <b>v2 + learned pre-filter <math>p1 \geq 0.1</math></b>                  | <b>3.84 (test)</b>  | <b>0.9263</b> | <b>–</b>  |

Every uploaded file passed the official validator. The gap between held-out (0.926) and public (0.902) is expected: the test set is about 15% France, a country absent from training, and 47% India, the harder training country (held-out

0.8950 for India vs 0.9469 for the US).

## 4.2 Error analysis (report split, 30,057 entities)

- **Where recall is lost.** Of 103,940 true pairs, 8,965 (8.6%) never reach the top-30 candidates and 6,266 (6.0%) are retrieved but rejected by the model. There are 1,837 false positives; 1,521 of 1,673 singletons are correctly left empty.
- **Share of lost F0.5:** 69% partial recall on entities with matches, 24% entities with matches that get an empty prediction, 7% singletons given a false match.
- **False positives:** near-identical distractors with the same name at a slightly different number (*1572 vs 1582 Hardee Street*, door *16-5-1 vs 16-5-3*); a different business at the same address; one-character name changes (*LFZ Foods vs LZ Foods*) at an identical address.
- **False negatives:** candidates with an empty address; true matches with an unrelated trade name or a name in Kannada or Devanagari script; heavy typos (*CHENNAI PRTEDECTON*); street numbers that differ between sources (*1557 vs 1330 Bunce Road*).

## 4.3 Compliance

- Only the provided data is used: no geocoding, business registries, external APIs, web data or augmentation. The abbreviation and legal-form lists are short, generic, hand-written rules.
- Dependencies are permissively licensed: LightGBM and rapidfuzz (MIT); pandas, NumPy, SciPy and scikit-learn (BSD; scikit-learn is used only for feature hashing). No GPL code. Model size is a few MB, far below 8B parameters.
- `candidate_pairs.tsv` is exactly the set the final model scored, and matches are a subset of it.

## 4.4 Reproducibility and efficiency

- `python src/main.py train` then `python src/main.py predict` reproduce the models and both output files; `utils/validate_submission.py` is the official validator, unchanged. All randomness is seeded.
- Runs on one laptop (Windows 11, 14 cores, 31 GB RAM): training 31 min; test prediction 58 min (blocking 12 min, scoring 39 min over 51.9M retrieved pairs).
- Memory-conscious engineering: sources are read and normalized in 1M-row chunks; each country's index is freed before the next is built; features are computed in ~3M-pair chunks that never split an entity's candidates (the gap and group features are per entity).
- Output files are written with LF line endings and no quoting, since the official scorer strips only `\n`.

## 4.5 Limitations and next steps

- **Blocking recall is the main ceiling:** 8.6% of true pairs are never retrieved. Transliteration-aware keys for native-script names and character n-gram keys for heavy typos are the most promising additions.
- France is unseen in training. Normalization already drops common French legal forms and articles (SARL, SAS, EURL, de, la), and the features are language-agnostic, but the model and thresholds are calibrated on US and India data only.
- Distractor records with a near-identical address need finer number handling, for example comparing structured door and unit numbers instead of token sets.